

cachectl — cache-management CLI operator guide

cachectl — cache-management CLI operator guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:** Restored end-user feature (regression #50). The 368-line CLI was accidentally deleted in commit 6ec58ef and is restored under the non-colliding name `cachectl`; live-verified against the running rootless-Podman proxy + `sh -n/bash -n parse-clean`.

Companion (§11.4.18) to the in-source documentation block at the top of the `cachectl` script.

Overview

`cachectl` manages, inspects, and invalidates the proxy's on-disk cache (`$CACHE_DIR`, default `./cache` with `squid/` and `streaming/` subtrees). It is the operator-facing cache tool documented in `README.md`, `USER_GUIDE.md`, `docs/CACHE.md`, and `docs/TROUBLESHOOTING.md`.

It was originally shipped as a tracked file named `cache`. Because `CACHE_DIR` is `$PROJECT_ROOT/cache` and `.gitignore` ignores `cache/` (the runtime **data** directory), once the runtime materialised that data directory it collided at the same path with the tracked `cache` **file**, and a broad `git add` recorded the file as deleted (regression #50). It is restored as `cachectl` (extensionless, matching the `./start` convention) so the executable and the gitignored `cache/` data directory coexist without collision.

Prerequisites

- `lib/container-runtime.sh` (sourced for `log`, `load_environment`, `init_runtime`, `is_container_running`, and `CACHE_DIR`).
- `bash`, `du`, `find`, `sort`, `cut`, `wc`.
- Rootless **Podman** (§11.4.161) for the invalidate squid-rotate step; the read-only commands need no container runtime.

Usage examples

```
./cachectl stats           # cache statistics (size, file/dir
                           counts, oldest/newest)
./cachectl size            # size breakdown by top-level dir +
                           file-size histogram
./cachectl list            # cache directory structure (du -sh per dir, depth 3)
./cachectl invalidate      # remove stale (> CACHE_MAX_AGE_DAYS)
                           files + rotate squid
./cachectl clear -f        # force-clear all cached content,
                           recreate squid/streaming
./cachectl trim 30         # trim cache down to 30 GB (oldest-
                           first)
./cachectl warmup          # (placeholder – not implemented yet)
./cachectl --help         # usage
```

Options: `-h|--help`, `-v|--verbose` (sets `LOG_LEVEL=debug`), `-f|--force` / `-y|--yes` (skip confirmation prompts).

Relevant environment (read by `load_environment`): `CACHE_DIR`, `CACHE_MAX_SIZE_GB` (default 50), `CACHE_MAX_AGE_DAYS` (default 30), `LOG_LEVEL`.

Edge cases

- **Rootless-Podman partial read (expected, not an error).** Under rootless Podman, squid runs as a remapped high subuid and creates `cache/squid/00..0F` hash sub-directories the host user cannot traverse, so `du/find` legitimately exit non-zero ("Permission denied") on those sub-dirs. `cachectl` therefore disables `pipefail` (after sourcing the lib, which sets it) so the `stat/list` commands report the correct **host-side partial view** instead of aborting mid-output. The container-owned sub-dirs are **never** `chmod/chown`-ed (§11.4.133 — squid FATALs on permission changes).
- **CONTAINER_RUNTIME initialised in main().** `invalidate`'s `squid-flush` case `"$CONTAINER_RUNTIME"` and `is_container_running` require the runtime to be set; `main()` calls `init_runtime` (tolerating a missing `compose` binary so read-only commands still work) and falls back to `detect_container_runtime`.

- **Destructive commands** (clear, invalidate, trim) honour -f/-y; without them they prompt for confirmation. Always test destructive behaviour against an **isolated throwaway** `CACHE_DIR`, never the live `./cache` (§11.4.133 / §9.2).
- **Missing cache directory** → `stats/size` print a "not found" hint and return non-zero; run `./init`.
- **warmup** is a documented placeholder (Warmup not implemented yet).

Internal behaviour

- `#!/usr/bin/env bash, set -eu; pipefail` deliberately disabled after sourcing `lib/container-runtime.sh` (see Edge cases). `bash -n parse-clean`.
- `trim` uses the `set -e-safe removed=$((removed + 1))` counter form; `((removed++))` returns exit 1 when `removed` is 0 and would abort the trim loop under `set -e` (§11.4.1).
- `invalidate` removes files older than `CACHE_MAX_AGE_DAYS`, prunes empty directories, trims to `CACHE_MAX_SIZE_GB` if over budget, then (when `proxy-squid` is running) issues `squid -k rotate` via the detected runtime.
- Read-only commands (`stats/size/list`) touch no container and no data plane; resources are shell + `du/find` only, within the §12.6 host ceiling.

Related

- `lib/container-runtime.sh` — sourced `runtime/log/env` library.
- `tests/regression/cache_cli_present_test.sh` — §11.4.135 standing regression guard (present + executable + parseable + all documented subcommands), with §11.4.115 `RED_MODE` polarity; registered in `tests/run-tests.sh` `test_regression_guards()`.
- `tests/comprehensive-test.sh` — exercises `stats/size/list` against the live cache.
- `README.md`, `USER_GUIDE.md`, `docs/CACHE.md`, `docs/TROUBLESHOOTING.md` — user-facing cache documentation.
- Constitution §11.4.1 / §11.4.18 / §11.4.108 / §11.4.111 / §11.4.124 / §11.4.133 / §11.4.135 / §11.4.161 / §12.6.

Last verified

2026-07-01 — `./cachectl stats|size|list` run against the live rootless-Podman proxy cache: all three print real cache data and exit 0; `clear/invalidate/trim` validated on an isolated throwaway `CACHE_DIR` (exit 0, correct removals); `sh -n/bash -n parse-clean`;

regression guard GREEN + RED + §1.1 mutation proven.
Evidence under qa-results/cachectl/ and qa-results/regression/
cachecli/.